# SYNTHOLOGY: Ontology-Based Data Generation for Neuro-Symbolic Reasoning

Vincent VAN SCHEPENDOM [1], Cas PROOST and Pieter BONTE
*Department of Computer Science, KU Leuven campus Kulak Kortrijk*

**Abstract.**
**Purpose:** Unlike traditional symbolic reasoners, which can operate directly on an ontology, neuro-symbolic reasoners require specific training data to learn *approximate* reasoning. Real-world knowledge graphs are too noisy and incomplete for this, leaving synthetic data as the only viable option. This work provides exactly that, generating the required high-quality training data, guided by *any* ontology.
**Methodology:** We present SYNTHOLOGY, an ontology-agnostic synthetic data generator for neuro-symbolic reasoning over OWL 2 RL. Our approach engineers information-dense, multi-hop training data. We also introduce proof-based negative sampling, which creates near-miss negatives that aren't superficially detectable. These two dataset properties require neuro-symbolic models to traverse the whole reasoning chain in order to correctly classify a target triple as inferred or not.
**Findings:** A neuro-symbolic reasoner trained on SYNTHOLOGY data approximates the predictions of a symbolic reasoner and substantially outperforms baselines trained on Unguided Deductive Materialization data. We find that proof-based negative sampling creates structurally-grounded near-miss negatives that are substantially harder to reject than random corruptions, and that training a neuro-symbolic reasoner on these proof-based negatives yields consistent improvements over both random and constrained corruption baselines on all learning-quality metrics. We show that our approach also scales to ontologies on which reasoner-based synthetic data generation pipelines fail to terminate.
**Value:** SYNTHOLOGY enables the training of neuro-symbolic reasoners without expensive data gathering. It is the first ontology-based synthetic data generator providing native multi-hop coverage, hard negatives, and derivation metadata. It scales to bigger ontologies where forward-chaining baselines fail, and is usable for both training, as well as benchmarking neuro-symbolic reasoners, given *any* ontology.

**Keywords.** Neuro-Symbolic AI, Approximate Reasoning, Synthetic Data Generation, Ontology-Agnostic, Negative Sampling, Knowledge Graph Reasoning

## 1. Introduction

Knowledge Graphs (KGs) and ontologies form the backbone of structured knowledge representation on the Semantic Web. The task of deriving new, implicit knowledge from these structures is broadly known as Knowledge Graph Reasoning (KGR), and is the

[1] Corresponding Author: Vincent Van Schependom, vincent.vanschependom@student.kuleuven.be

broader task this work is ultimately concerned with. Traditionally, KGR is performed by symbolic reasoners [1,2], which directly execute the ontology's rules with perfect soundness and completeness. While elegant, this approach hits a practical wall: symbolic reasoners are sensitive to noise and computationally expensive, making them infeasible on massive or messy real-world KGs [3].

Neuro-symbolic reasoning has emerged as the proposed alternative within the field of KGR [4]. Instead of executing the ontology's rules at inference time, a neural model is trained to imitate them. The resulting behaviour is called *approximate reasoning* [5]: the model predicts the entailments of the underlying knowledge base in a way that is scalable and tolerant to noise. This shift, however, comes with a critical prerequisite – and addressing it is the central focus of our work. A symbolic reasoner can be pointed straight at an ontology and immediately produce entailments, but a neuro-symbolic reasoner first has to be *trained* on a dataset that reflects the target ontology. Gathering such a dataset is often prohibitively expensive, or outright impossible, which means that, in the absence of suitable data, neuro-symbolic reasoners simply cannot be deployed at all.

Real-world datasets such as DBpedia or Freebase [6,7] are typically noisy or incomplete, making it difficult to distinguish genuine logical learning from statistical shortcuts. Synthetic data generators could solve this problem. Standard approaches typically utilize procedural generation of base facts [8,9], followed by materialization: a logical reasoner infers the to-be-predicted targets from the previously generated base facts, as in [10]. This methodology often produces structurally shallow graphs, because the generation is not ontology-aware. The resulting graphs contain a lot of noise, while the deep, multi-hop derivations that the model is actually meant to learn occur only by accident. On top of this, the resulting datasets lack challenging negative samples, which are critical for training neuro-symbolic models: random or trivial negatives are easy to reject from surface features alone, so the training signal collapses to pattern matching rather than reasoning.

To address this gap, we present SYNTHOLOGY, an ontology-driven synthetic data generator. Our novel generator can take *any* ontology using the supported subset of OWL 2 RL. In a backward-chaining manner, it purposefully engineers training data that inherently requires multiple hops to be correctly predicted. Crucially, it also generates hard negative samples based on the constructed proof trees. These negatives *look* plausible but are not logically entailed by the KB, forcing models to learn the underlying ontology structure rather than relying on superficial indicators. In summary, our main contributions are:

- **Synthology**: the first ontology-agnostic, backward-chaining synthetic data generator for OWL 2 RL, designed to guarantee deep multi-hop inference chains by construction. *Any* supported ontology can now be used to train a neuro-symbolic reasoner without expensive data gathering.
- **Proof-based negative sampling**: a method for constructing hard negatives directly from proof trees, producing near-misses that require genuine multi-hop reasoning to correctly classify.
- **Empirical evaluation**: a comparative study across two ontologies (Family Tree, OWL2Bench) demonstrating significant advantages in hop distribution, predicate coverage, negative sample quality, and scalability over Unguided Deductive Materialization baselines.

As a supporting deliverable, we also release an open-source PyTorch Lightning reimplementation of the Recursive Reasoning Network [11] with batched message passing, used as the evaluation model throughout this paper.

Section 2 introduces the necessary background, Section 3 reviews related work, Section 4 details SYNTHOLOGY's design, Section 5 describes the experimental setup, Section 6 reports results, and Section 7 discusses limitations and future work.

## 2. Background

This section establishes the formal vocabulary used throughout the paper. We first define knowledge graphs and ontologies, then introduce the Recursive Reasoning Network used as our evaluation model, and finally outline the role of negative sampling in training neural reasoners.

### *2.1. Knowledge Graphs and Ontologies*

We consider a Knowledge Graph as a collection $\mathcal{G}$ of concrete facts (assertions) about specific entities, referred to as the ABox. To provide semantics and enable reasoning, the KG is paired with an ontology $\mathcal{O}$ (the TBox). Together, the factual KG and the ontology form a Knowledge Base $\text{KB} = \langle \mathcal{G}, \mathcal{O} \rangle$.

For example, the triples $\langle \text{Alice}, \text{hasParent}, \text{Bob} \rangle, \langle \text{Bob}, \text{hasParent}, \text{Charlie} \rangle$ are explicit, *base facts* in the KG, while a rule like $\langle X, \text{hasParent}, Y \rangle \wedge \langle Y, \text{hasParent}, Z \rangle \Rightarrow \langle X, \text{hasGrandparent}, Z \rangle$ is part of the ontology $\mathcal{O}$. Both are present in the KB. Combining them, one can derive implicit facts, like $\langle \text{Alice}, \text{hasGrandparent}, \text{Charlie} \rangle$. These *inferred facts* are not explicitly stated in the ABox, but logically entailed by the ontology given the existing *base facts*. The *base/inferred* terminology will be used throughout this paper.

In this work, we focus on the OWL 2 RL profile [12] as the target rule language, though SYNTHOLOGY is built for extensibility to other profiles.

### *2.2. The Recursive Reasoning Network*

Throughout this paper, the *Recursive Reasoning Network* (RRN) [11] serves as the evaluation model. The RRN is trained to predict the likelihood $\mathbb{P}(\langle s, p, o \rangle = 1)$ of a triple being logically entailed by the provided base facts. It uses an iterative message-passing mechanism, allowing the model to capture complex, multi-hop logical dependencies. Crucially, the training data must contain sufficient evidence of these deep patterns – which is precisely what SYNTHOLOGY is designed to provide.

### *2.3. Negative Sampling*

Apart from deep chains, another critical factor in training neuro-symbolic reasoners (like the RRN) is negative sampling. Since KGs contain only positive facts, models must learn to distinguish true inferences from false ones by training against corrupted (and thus false) facts. We consider two standard corruption strategies: *random* corruption replaces the subject or object with an arbitrary individual, while *constrained* corruption additionally requires the replacement to satisfy the corrupted predicate's declared domain or

range. Recent reviews of negative sampling strategies in KGR emphasize that creating 'hard' negatives – structurally similar, plausible facts that are not logically entailed – is a critical element in training neuro-symbolic models, as it forces them to learn *nuanced* logical patterns rather than relying on trivial entity mismatches or superficial indicators [13].

## 3. Related Work

This section reviews existing approaches to generating synthetic data. We start with benchmark-based ABox generators and the "generate + materialise" pipeline that combines them with a symbolic reasoner, and then discuss alternative methods.

### *3.1. Benchmark-Based ABox Generators*

Several benchmarks for OWL reasoning provide a procedural ABox generator that creates synthetic data following a fixed schema. The Lehigh University Benchmark (LUBM) [8] populates a university domain – students, professors, courses, departments – at configurable scales. Its successor, the University Ontology Benchmark (UOBM) [9], extends LUBM to exercise additional OWL constructs. OWL2Bench, finally, provides a generator paired with an OWL 2-aligned schema, designed to stress all four OWL 2 profiles (including RL), once again over a university domain.

The problem is that each of these generators is tightly coupled to its underlying ontology: the type distributions, individual roles, and relation densities are hand-tuned to a specific schema, so the generator cannot be reused for an arbitrary user-supplied ontology. Furthermore, there is no base/inferred fact distinction, so we still need to create the targets that the neuro-symbolic model should predict.

### *3.2. Unguided Deductive Materialization*

To produce training targets, such a benchmark generator (or a fully random ABox) can be paired with a symbolic reasoner that materialises the deductive closure [10]. We refer to this two-step pipeline as *Unguided Deductive Materialization* (UDM): a procedurally or probabilistically generated set of base facts is first constructed – either tied to a fixed ontology or drawn at random – and a symbolic reasoner then materialises all logically entailed facts to form the training targets. Negatives are typically added via uniform random subject or object corruption [14].

The crucial issue with UDM is that generation *is not guided by the ontology*. The generator produces individuals and assertions from statistical templates, and the reasoner is invoked only afterwards. As a result, the generated data rarely satisfies the conditions required to fire long rule chains: predicates that depend on the simultaneous presence of multiple structurally compatible facts produce few, if any, inferences, while shallow schema propagations (`rdf:type` assertions, `rdfs:subClassOf` closures) dominate the closure. Constructs that require *multi-hop reasoning* are therefore systematically under-represented in the resulting training data, regardless of how many base facts are generated, and neural models trained on this output are prone to *shortcut learning* [15].

A second concern is scalability. Forward-chaining reasoners like Apache Jena [2] typically use semi-naive evaluation, which iteratively saturates the ABox until no new

facts can be derived. Although each iteration is bounded, the materialised closure must be retained in memory throughout. In practice, this causes materializers to fail at production-scale ontologies – a ceiling we refer to as the *Reasoning Wall* and characterise empirically in Section 6.3.

### *3.3. Ontology-Specific ASP Generators*

Rather than relying on a benchmark generator + reasoner pipeline, some prior work uses Answer Set Programming (ASP) to encode the generation process directly. The most prominent example is the RRN paper [11], which provides an ASP program tailored to a single ontology (the Family Tree domain) that produces both base facts and inferred targets in one pass. While this couples generation to logical structure (unlike UDM), the ASP program must be hand-written for each new ontology, making extension to a different domain non-trivial. The approach also provides no native mechanism for proof-based negative sampling.

### *3.4. LLM-Based Generators*

Large Language Models have recently been explored for KG construction, generating triples from natural-language descriptions of a domain. While promising for naturalistic verbalisation, LLM outputs lack formal correctness guarantees and are thus not always suited for generating the logic-consistent data on which a neuro-symbolic reasoner can be supervised [16].

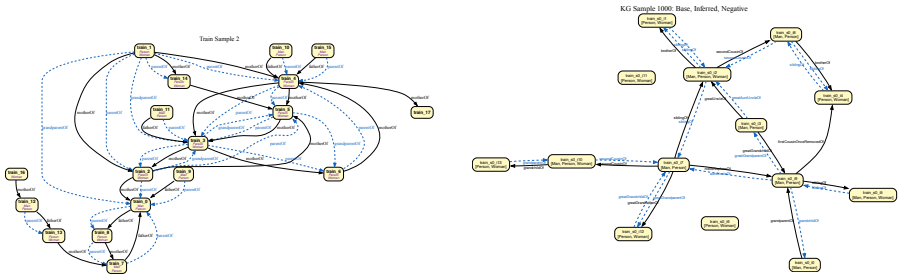
## 4. Methodology: Targeted Data Generation

To address the shortage of neuro-symbolic reasoning-focused training data, we introduce SYNTHOLOGY. Unlike standard pipelines, our generator engineers an ABox through ontology-guided backward-chaining proof search. In this way, we produce base facts driven by logical *necessity* rather than statistical randomness. Figure 1 illustrates the qualitative difference: starting from the same Family Tree ontology, SYNTHOLOGY produces a connected graph rich in deep multi-hop inferences, while the unguided baseline produces a structurally shallow graph dominated by trivial single-hop derivations. The full codebase – generator, ontologies, and experimental setups – is open-source[2].

SYNTHOLOGY accepts ontologies expressed in a supported subset of OWL 2 RL[3] and executes a three-phase pipeline. A *parsing* phase first compiles the ontology into a set of executable Horn rules $\mathcal{R}$, a schema $\mathcal{S}$ (classes, relations, attributes), and constraints $\mathcal{C}$. The *generation* phase (Algorithm 1) then drives a sample-level loop, in which backward-chaining proof search constructs proof trees from grounded rule conclusions. Finally, in the *validation* phase, each candidate graph is checked against $\mathcal{C}$ and injected with negative samples (Section 4.3).

[2] `https://github.com/schependom/synthology`

[3] RDFS schema axioms (`rdfs:subClassOf`, `rdfs:subPropertyOf`, `rdfs:domain`, `rdfs:range`), OWL 2 property axioms `owl:inverseOf`, `owl:propertyChainAxiom`, `owl:SymmetricProperty`, `owl:TransitiveProperty`, and `owl:ReflexiveProperty`, & constraints `owl:IrreflexiveProperty`, `owl:AsymmetricProperty`, `owl:FunctionalProperty`, and `owl:disjointWith`




(a) *KG generated by* SYNTHOLOGY.
The KG contains a rich variety of multi-hop inference paths, with a balanced distribution of easy, medium, and hard samples. The graph is structurally coherent and rich in deep logical chains.

(b) *KG generated by the UDM baseline.*
The KG is structurally shallower than the SYNTHOLOGY graph, with only single-hop, trivial, inferences.

**Figure 1.** Toy knowledge graphs generated by SYNTHOLOGY and the UDM baseline (random ABox). Black edges represent base facts, blue edges inferred facts. Both graphs are generated from the same Family Tree ontology and `rdf:type` triples were omitted for clarity. This figure is intended as an illustrative comparison; quantitative evidence of the structural gap is provided in Section 6 and Table 2.

This pipeline is modular by design: new OWL constructs can be supported by adding parser handlers and rule templates, without changing the chainer itself. Users can trade structural properties against generation cost via a set of configurations – the recursion cap, the individual reuse probability $\pi_{\text{reuse}}$, the depth-bucket weights, etc. – and the framework supports *sweeps* for systematic optimisation of these parameters.

### *4.1. Sample Construction*

Data construction (Algorithm 1) assembles one knowledge graph per iteration. A random subset of rules $\mathcal{R}' \subseteq \mathcal{R}$ is selected. For each selected rule $r \in \mathcal{R}'$, $k_r$ independent groundings of its head are created and passed to the backward chainer, which returns a set of proof trees $P_r$. We therefore refer to the $k_r$ independent groundings of the rule head as *proof roots*. A configurable number of proofs is chosen per rule (depth-stratified or signature-based, see below) and merged into a global *proof map* $M$. This map links every atom appearing in the selected proofs with the list of derivations that produced it. $M$ is then materialised into a candidate KG, extended with class-membership assertions implied by the domain and range declarations of its triples, validated, and injected with negative samples.

The proof search space is potentially combinatorial: each premise may admit multiple sub-proofs, and combining them requires a Cartesian product. This repeats recursively down the tree. SYNTHOLOGY therefore uses *bounded* proof engineering: the global proof depth, the number of times a recursive rule may be re-applied, and the number of proof trees retained per atom are all capped.

A key mechanism for producing connected, non-trivial graphs is the *individual reuse pool*. When the chainer binds an unbound variable, it reuses an existing pool member with probability $\pi_{\text{reuse}}$, subject to domain/range compatibility. Otherwise, a fresh individual is created and added to the pool. This controlled individual sharing causes derivations to overlap, resulting in interconnected graphs rather than isolated derivation trees. The

reuse probability is exposed as a configurable hyperparameter, allowing users to trade graph density against derivation diversity for their target ontology.

**Algorithm 1** Sample Construction

**Require:** Rules $\mathcal{R}$, chainer $\mathcal{B}$, target count $N_S$, configuration $\Theta$
**Ensure:** Sample set $\mathcal{D}_S$

```
1:  D_S ← ∅
2:  while |D_S| < N_S and attempt budget not exhausted do
3:      Reset proof map M ← ∅ and individual pool
4:      Select rule subset R' ⊆ R
5:      for each rule r ∈ R' do
6:          Sample root count k_r ~ U[k_min, k_max]
7:          P_r ← ∅
8:          for i = 1 to k_r do
9:              Ground head of r with pool/fresh individuals, obtaining goal g
10:             P_r ← P_r ∪ PROVE(g, 0, ∅)
11:         end for
12:         if P_r = ∅ then continue
13:         end if
14:         P̃_r ← retain up to n_r proofs from P_r
15:         Merge the proof map contributions of P̃_r into M
16:     end for
17:     if M = ∅ then continue
18:     end if
19:     Convert M to a candidate KG; apply schema completion
20:     Perform validation checks
21:     Add negatives per configured strategy (Section 4.3)
22:     D_S ← D_S ∪ {KG}
23: end while
```

### *4.2. Backward-Chaining Proof Search*

For each of the candidate rules $p_1 \wedge ... \wedge p_n \rightarrow h$, the chainer first checks the per-rule recursion cap and it prunes the branch if the rule's head $h$ is the inverse of the parent predicate (to eliminate infinite inverse loops). Then, the chainer unifies the goal $g$ with $h$, binds remaining variables via the reuse pool, and recursively proves the grounded premises $p_1, ..., p_n$. At most $P_{\max}$ combinations of premise proofs are withheld per goal. If no rule succeeds, the chainer falls back to a base proof, ensuring every reachable goal predicate admits at least one derivation.

To counteract the dominance of shallow schema proofs, SYNTHOLOGY supports *depth-stratified sampling*, which distributes withheld proofs across easy ($d \leq 1$), medium ($2 \leq d \leq 4$), and hard ($d \geq 5$) buckets using normalised weights. An optional *signature sampling* step additionally groups proofs by their structural signature – the multiset of rule names in the tree – and keeps one representative per signature, reducing near-isomorphic proofs. The resulting dataset preserves a strict base-versus-inferred distinction, together with per-triple derivation metadata.

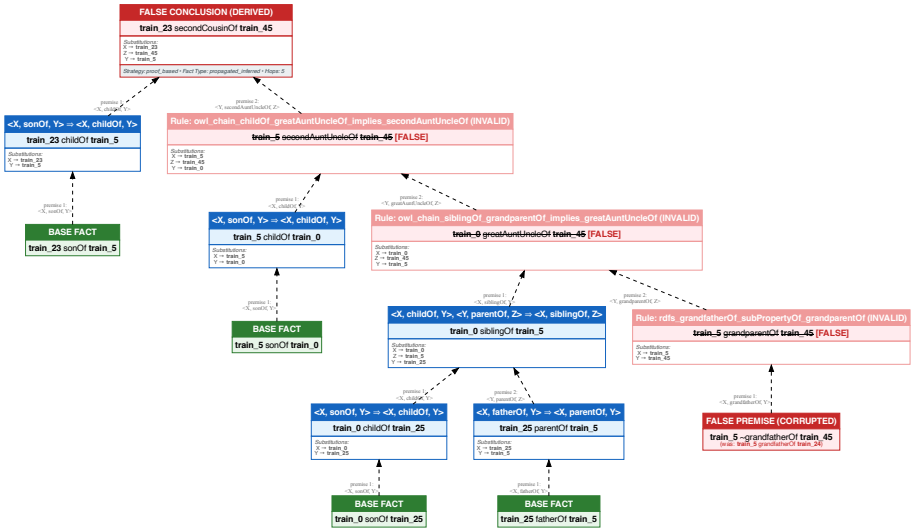



**Figure 2.** A corrupted proof tree. The base fact $b = \langle 5, \text{fatherOf}, 24 \rangle$ is corrupted to $\tilde{b} = \langle 5, \text{fatherOf}, 45 \rangle$, which propagates up the proof tree to the logically invalid inferred 5-hop fact $\tilde{t} = \langle 23, \text{secondCousinOf}, 45 \rangle$.

### *4.3. Proof-Based Negative Sampling*

Random and constrained corruption (Section 2.3) both operate on a single triple in isolation and are therefore blind to the derivation that produced it. While current negative sampling literature typically categorizes methodologies into static, dynamic, or purely adversarial approaches [13], SYNTHOLOGY introduces *proof-based* negative sampling, which targets *the logical support* of inferred facts by corrupting a base fact and propagating the error upward along its proof tree.

Let a positive target triple $t^+$ have at least one proof tree $\pi$, and let $\mathcal{B}(\pi)$ be the base facts used to derive $t^+$. The proof-based strategy samples a base atom $b \in \mathcal{B}(\pi)$ and perturbs its subject or object, yielding $\tilde{b}$. This substitution is propagated through the derivation chain, producing a logically flawed goal atom $\tilde{t}$. Crucially, the corruption only breaks one leaf of the proof: the rest of the tree remains structurally valid, so $\tilde{t}$ is backed by a near-perfect derivation – an exceptionally difficult negative. Figure 2 shows one such corrupted proof tree generated by SYNTHOLOGY on the Family Tree ontology, an OWL 2 RL translation of the `familytree` ASP program from [11].

A corrupted pair $(\tilde{b}, \tilde{t})$ is accepted only if $\tilde{t}$ does not coincide with an existing positive in the graph. Because the input graph contains no evidence for $\tilde{b}$, no valid proof of $\tilde{t}$ exists within it, and the model must therefore detect the broken logical chain from the remaining evidence. The model is trained to predict $\mathbb{P}(\tilde{t}) \approx 0$ in this case. To keep the training distribution unskewed, a single corrupted proof tree currently contributes at most one negative, being only the root goal atom.

To meet the configured negative ratio in finite time, the generator can fall back to constrained corruption when proof-based construction fails. In practice, however, fallback rarely occurs for dense ontologies (such as our Family Tree ontology) and the dataset consists almost entirely of hard proof-based negatives. SYNTHOLOGY additionally exposes a *mixed* strategy as a configurable option, blending proof-based negatives

with constrained corruptions in a user-specified ratio. The intuition is that simpler structural negatives could provide complementary signal alongside the harder proof-based ones; whether this hypothesis holds in practice is examined in Experiment 1.

Finally, it is worth noting that the candidate pool for corruption contains *all* positive inferred triples, not just top-level entailments. An intermediate fact has its own proof tree and can be corrupted in the same way as an inferred (root) fact. The depth of each generated negative therefore equals the depth of the proof tree from which it was derived, and the dataset naturally contains negatives at every level of the inference hierarchy.

## 5. Experimental Setup

To validate SYNTHOLOGY, we evaluate the quality of the generated data in two ways. Firstly, the characteristics of the generated data (e.g. hop distribution), as well as the generation metrics (e.g. elapsed time) are analyzed. Secondly, the RRN [11] is trained on data generated by both SYNTHOLOGY and UDM, after which its predictive performance is analyzed and compared.

### *5.1. The Unguided Deductive Materialization (UDM) Baseline*

We treat UDM as a family of approaches rather than a single fixed dataset: either (1) a randomly, or (2) a procedurally generated ABox (for example with a benchmark) is first constructed under the constraints of the given input ontology. After the ABox is generated in one of the two ways, a symbolic reasoner – in our case Apache Jena – materialises its deductive closure to obtain positive targets. Negatives are then added via 1:1 random subject/object corruption applied uniformly to all positive triples (both base and inferred).

For Experiment 2, we use the random ABox variant (1): we create an individual pool per sample, assign each individual a random set of ontology classes, and create object-property facts by uniformly sampling predicates under the schema constraints, until the configured base-fact budget is reached. Experiment 3 uses the procedural variant (2), instantiating the ABox via the OWL2Bench Java generator. The Jena reasoning profile differs per experiment: Experiment 2 uses a lightweight profile focused on property-chain inferences (avoiding broad shallow derivations such as disjointness, in order to isolate deep multi-hop reasoning), while Experiment 3 uses a more expressive profile that supports the richer OWL 2 RL semantics required by the OWL2Bench ontology.

To ensure a *matched budget*, we balance both the base-fact count and the positive inferred-target count between the two pipelines. Both generators are configured to exceed the desired budget, after which a post-hoc balancing script trims samples in generation order until the cap is reached. In practice, the resulting budgets agree to within $\sim$10%; the residual mismatch is discussed in Section 7. Because Jena does not expose proof-depth distributions natively, we compute hop depth via a custom derivation-tracing pass over inferred facts, which introduces additional runtime overhead in Experiment 2 beyond closure computation itself.

For Experiment 3, derivation logging is disabled during the full-scale baseline generation ($u = 14$ universities) due to its memory cost (Section 6.3). Instead, derivation logging is enabled only in a small run ($u = 3$ universities), whose hop distribution is used

as a representative proxy for the full-scale baseline. This substitution is valid because OWL2Bench university ABoxes are structurally independent fragments over the same TBox: the same rules fire at the same proof depths regardless of how many universities are instantiated.

All exact SYNTHOLOGY configurations and the full set of RRN hyperparameters used for each experiment are provided in the open-source repository.

### *5.2. The Evaluation Model: Recursive Reasoning Network*

Because the original RRN publication did not provide an official source-code release, we reimplemented the architecture from scratch. To fully utilize GPU parallelization, we grouped message passing updates into batches. The reimplementation preserves the original learning objective and matches the reference accuracy on the Family Tree dataset closely, while reducing training time drastically compared to the unbatched version. As noted in Section 7, we did not invest more time than required in hyperparameter tuning for the RRN, so the reported results should be interpreted as a lower bound on the architecture's performance on these datasets.

The RRN outputs a probability score $P := \mathbb{P}(\langle s, p, o \rangle) \in [0, 1]$ indicating the likelihood that the target fact $\langle s, p, o \rangle$ is entailed by the base facts. We evaluate it as a binary classifier and report PR-AUC and AUC-ROC as threshold-independent metrics, alongside F1-score and the False Positive Rate (FPR) at threshold $P > 0.5$. We treat PR-AUC and FPR as the primary metrics, because the central claim of this paper concerns robustness against logically false near-miss negatives – a property that PR-AUC and FPR penalise more aggressively than AUC-ROC. Per-class accuracy ($\text{Acc}^+$, $\text{Acc}^-$, Recall) is reported for completeness in Table 1, where it helps diagnose the conservatism/permissiveness profile of each model.

### *5.3. Experiment 1: Negative Sampling Strategy Evaluation*

Using the Family Tree ontology, we generate SYNTHOLOGY variants that differ only in the negative-sampling strategy: standard random corruption, constrained random corruption, pure proof-based corruption, and a mixed strategy (blending 50% constrained and 50% proof-based corruption). Each variant consists of $N_{\text{train}} = 2000$ training, $N_{\text{val}} = 500$ validation, and $N_{\text{test}} = 500$ test knowledge graphs. A separate RRN is trained on each variant and evaluated on the same held-out test set, rich in near-miss targets.

### *5.4. Experiment 2: Multi-Hop Reasoning Quality*

To test multi-hop reasoning capability, we use the same Family Tree ontology, since it enables deep inference chains. An RRN trained on the UDM baseline is compared against an RRN trained on SYNTHOLOGY data under the matched-budget protocol described in Section 5.1, configured to target a comparable input-graph size ($\sim$200K input facts per dataset; see Table 2). Evaluation is performed on a frozen, held-out test set generated once by SYNTHOLOGY with elevated recursion/depth limits and then reused unchanged for both models.

We note that *completely* random ABox generation is deliberately the *worst case* for the UDM pipeline: the reasoner receives unstructured inputs, and this is therefore the hardest setting in which to produce deep inference chains. An ontology-aware ABox,

**Table 1.** RRN predictive relational performance for Experiment 1 and Experiment 2.

| Exp. | Dataset | PR-AUC (↑) | AUC-ROC (↑) | FPR (↓) | F1 (↑) | $Acc^+$. (↑) | $Acc^-$. (↑) | Recall (↑) |
|---|---|---|---|---|---|---|---|---|
| 1 | Random | 50.7% | 0.770 | 20.9% | 0.527 | 58.7% | 78.8% | 58.7% |
| | Constrained | 53.6% | 0.830 | 37.7% | 0.606 | 93.5% | 62.1% | 93.5% |
| | Proof-based | 54.6% | 0.843 | 36.2% | 0.628 | 96.0% | 63.4% | 96.0% |
| | Mixed | 46.6% | 0.730 | 63.8% | 0.490 | 94.9% | 36.6% | 94.9% |
| 2 | UDM | 38.8% | 0.594 | 54.1% | 0.457 | 69.7% | 45.9% | 69.7% |
| | SYNTH. | 60.3% | 0.825 | 34.6% | 0.636 | 86.3% | 65.1% | 86.3% |

constructed with handcrafted heuristics over the statistical distribution of types and relations, would close part of the gap, but every such heuristic is ontology-specific and is therefore not generalizable. Experiment 3 re-runs the comparison against a strong, benchmark-provided ABox generator to test the gap from the opposite direction.

### *5.5. Experiment 3: Generalization to Complex Ontologies*

To demonstrate generalization to more complex ontologies, and to validate the ontology-agnostic claim of SYNTHOLOGY, we evaluate both generators – ours and the UDM approach – on the OWL2Bench benchmark ontology. For the UDM baseline, the OWL2Bench Java generator instantiates a university ABox for $u$ universities, Apache Jena materialises its deductive closure, and the resulting graph is partitioned via BFS – initialized at random individuals and capped at $k$ individuals per subgraph – into sample-sized knowledge graphs. Inferred triples that span different subgraphs are discarded.

## 6. Results and Discussion

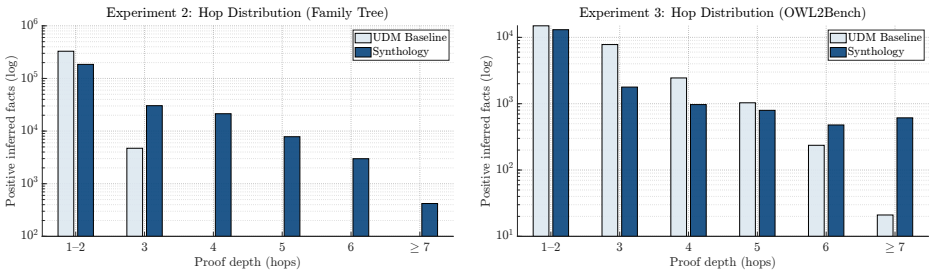
### *6.1. Experiment 1: Negative Sampling Strategy Evaluation*

Table 1 reports RRN performance for the four negative-sampling strategies. Proof-based negative sampling achieves the best score on five of seven metrics – PR-AUC, AUC-ROC, F1, $Acc^+$, and Recall – with random corruption only winning on FPR and negative triple accuracy ($Acc^-$). As we explain below, however, these two apparent wins are *not* evidence of better reasoning. On the metrics that capture genuine ranking and classification quality, proof-based wins consistently. The improvement over random is substantial across the board. The gap to constrained corruption is smaller but consistent on these same five metrics.

The two metrics on which random corruption wins (FPR and $Acc^-$) both reward a model for rejecting triples, and reading them in isolation is misleading. The random-trained model achieves $Acc^-$ of 78.8% but only 58.7% $Acc^+$. This reflects a classifier that learned *conservatism* rather than reasoning: it rejects most uncertain triples, which trivially produces a low FPR and a high $Acc^-$, but at the cost of missing a large fraction of true entailments. In contrast, both constrained and proof-based models exhibit the inverse profile – high positive accuracy (∼93–96%) with moderate negative accuracy (∼62–63%) – indicating that they successfully identified most true entailments, but occasionally accepted logically false near-misses, since these are hard to learn by design.

The modesty of the proof-based advantage over constrained corruption could be due to the architecture of the RRN. The iterative message-passing architecture is designed to perform well on multi-hop relational inference tasks. Because the model explicitly propagates evidence along edges for $k$ iterations, it can partially compensate for a less informative training distribution. The absolute size of the proof-based advantage at fixed hyperparameters should therefore be read as a property of the architecture-data interaction rather than an upper bound on what proof-based negatives can deliver.

The mixed strategy performs worst overall, falling below even the random baseline on PR-AUC, AUC-ROC, and F1, and yielding a negative accuracy of only 36.6%. The culprit is training-signal inconsistency: within a single batch the model encounters negatives whose detection requires different reasoning (shallow type-compatibility checks and deep proof-chain tracing) which prevents a stable decision boundary from being established. Rather than benefiting from the complementary signals, the network oscillates between two incompatible objectives and converges to an aggressive positive bias. This bias is reflected in Table 1: the mixed strategy has a large false positive rate of 63.8%, the worst of all four strategies. As discussed in Section 8, a curriculum-based blending schedule may implement the complementary-signal method better than the naive 50/50 mix.




(a) Experiment 2: *Family Tree*.
SYNTHOLOGY outperforms the UDM baseline, producing a deep hop distribution, in contrast with the baseline, which produces 99.3 % 1–2 hop facts and only 4,716 facts with $\geq 3$ hops.

(b) Experiment 3: *OWL2Bench*.
SYNTHOLOGY creates a deeper hop distribution than the UDM baseline, which gets a 'head start' because of the sophisticated OWL2Bench ABox (Section 6.3).

**Figure 3.** Distribution of positive inference hop depths in the SYNTHOLOGY and UDM datasets for Experiment 2 (Family Tree) and Experiment 3 (OWL2Bench). The Experiment 3 UDM baseline distribution is measured from the 2-university run, used as a proxy because full-scale derivation logging is infeasible (Section 5.1).

### *6.2. Experiment 2: Multi-Hop Quality*

Despite matched generation budgets, the UDM baseline and SYNTHOLOGY datasets show stark differences in both reasoning complexity and structural composition. Table 2 summarises the topological differences. Both methods produce comparable input-graph sizes (225K vs. 206K input facts, an 8.4% gap) and an almost identical total target volume ($\sim$640K). The UDM baseline produced a larger sheer number of Inferred Targets (333K vs. 249K for SYNTHOLOGY). However, as illustrated in Figure 3a, this volume is deceptive: the UDM baseline produced an overwhelming majority of its inferences at depth $d \leq 2$, with fewer than 1% of its facts requiring paths of $d \geq 3$ (only 4,716 inferences). In contrast, SYNTHOLOGY engineered a robust distribution of deep multi-hop proofs, creating 60,310 inferences at $d \geq 3$ and sustaining logical chains up to $d = 7$. The

**Table 2.** Comparison of topological dataset statistics across generation methodologies. While the overall generation budgets (Total Targets) are matched to $\sim 640k$ triples, and the context graph sizes (Graph Edges) are equalized within a 10% margin via sample trimming, the internal distribution of Base versus Inferred Targets inherently diverges due to the structural differences produced by forward versus backward chaining.

| Dataset | Method | Avg #/KG | | Input | Base Targets | | | Inferred Targets | | | Ratio |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | *Base* | *Inf.* | *Facts* | *Total* | *Pos* | *Neg* | *Total* | *Pos* | *Neg* | $\hat{\rho}_{\pm}$ |
| Family Tree *exp. 2* | UDM | 187.9 | 532.8 | 225k | 306k | 159k | 147k | 333k | 160k | 173k | 0.998 |
| | SYNTHOLOGY | 172.3 | 532.8 | 206k | 391k | 207k | 184k | 249k | 113k | 136k | 0.998 |
| OWL2Bench *exp. 3* | UDM | 67.4 | 210 | 14.5k | 18.8k | 6.1k | 12.7k | 26.4k | 16.7k | 9.7k | 1.020 |
| | SYNTHOLOGY | 252 | 824 | 13.9k | 27.7k | 13.9k | 13.8k | 17.6k | 9.0k | 8.6k | 1.020 |

**Table 3.** Comparison of computational time breakdown across generation methodologies. *Base Gen.* denotes the initial procedural or random ABox instantiation. *Inference* captures the Java-based forward-chaining materialization (Jena) or backward-chaining search (SYNTHOLOGY). *Overhead* includes BFS subgraph extraction, derivation-tracing for hop-depth computation, and negative sampling mechanics.

| Dataset | Method | Base Gen. | Inference | Overhead | Total Time |
|---|---|---|---|---|---|
| Family Tree *exp. 2* | UDM | <1 min | 26 min | 2 min | 30 min |
| | SYNTHOLOGY | / | 10 min | 1 min | 11 min |
| OWL2Bench ($u = 3$) *exp. 3* | UDM | <1 min | <1 min | 4 min | 6 min |
| | SYNTHOLOGY | / | 2 min | <1 min | 2 min |

UDM's higher inferred target count is thus simply the result of a combinatorial explosion of shallow, trivial inferences, whereas SYNTHOLOGY's budget is allocated towards deeper derivations.

Furthermore, the UDM baseline's unguided generation entirely failed to materialize inferences for 18 complex (e.g. `secondUncleOnceRemovedOf`) and gender-specific relations, resulting in exactly zero inferred targets for these predicates. This highlights a critical flaw in relying on unguided materialization: structurally demanding or highly constrained logical derivations occur only incidentally, making the UDM dataset systematically deficient. SYNTHOLOGY achieved complete predicate coverage, confirming that it proactively constructs the prerequisite evidence (base facts) for all defined logical goals (inferred facts).

The consequence of these structural differences is reflected in Table 1: the RRN trained on SYNTHOLOGY data outperforms the UDM-trained RRN on every reported metric. PR-AUC improves by 21.5pp (38.8% $\rightarrow$ 60.3%), AUC-ROC by 0.23 (0.594 $\rightarrow$ 0.825), and FPR drops by 19.5pp (54.1% $\rightarrow$ 34.6%). This is a substantially larger gap than any of the negative-sampling-only differences in Experiment 1, which is consistent with the larger structural change at play here: Experiment 1 varies only the negative-sampling strategy on top of the same multi-hop positive distribution, while Experiment 2 varies both the positive distribution *and* the negative-sampling strategy simultaneously.

### *6.3. Experiment 3: Generalization to Complex Ontologies*

*Hop Distribution.* Unlike the random ABox generation used in Experiment 2, the UDM baseline in Experiment 3 utilizes the OWL2Bench Java generator to procedurally instantiate realistic university structures. The baseline therefore produces a naturally deep and

**Table 4.** The Reasoning Wall. At production scale, UDM's forward-chaining approach fails to terminate, while SYNTHOLOGY's independent backward-chaining scales linearly in the number of requested samples.

| Scale | UDM Baseline (Jena) | SYNTHOLOGY |
|---|---|---|
| $u = 3$ | 6 min | 2 min |
| $u = 14$ | **OOM** after 7.5 h | 70 min (1M targets) |
| Memory | polynomial in $n$<br>cumulative | constant in $n$<br>per-sample |

highly connected ABox before any materialization occurs, giving the forward-chainer a stronger starting point. Despite this favourable setup, SYNTHOLOGY still produces a deeper hop distribution. Figure 3b shows that it creates a heavier tail of deep inferences than the benchmark-seeded baseline, demonstrating its ability to construct complex inferred facts even on richer schemas.

*Dataset Topology.* Table 2 shows that the structural patterns observed in Experiment 2 persist at a larger scale. Both generators were constrained to a nearly identical input graph size (14.5K vs. 13.9K input facts) and produced an equivalent total target count ($\sim$45K). The internal split, however, differs: UDM produces 26.4K inferred targets against 18.8K base targets, while SYNTHOLOGY produces 17.6K inferred against 27.7K base. Because the OWL2Bench generator provides a structurally rich ABox out-of-the-box, the subsequent forward-chaining triggers a wave of shallow ontological rule executions that dominates UDM's inference budget. SYNTHOLOGY's backward-chaining instead allocates the budget to constructing the specific base evidence required to support its targeted proofs, resulting in a leaner set of inferred targets that – as Figure 3b shows – carries a heavier deep-hop tail.

*Scalability: the Reasoning Wall.* A final practical question is how each generator scales with ontology size. UDM and SYNTHOLOGY are designed to do somewhat different jobs at the algorithmic level: UDM materialises the *full* deductive closure of an ABox, while SYNTHOLOGY samples $N$ *independent* proof trees. This difference is precisely what makes SYNTHOLOGY attractive when one wants a fixed-size training set rather than every entailed fact. UDM iteratively saturates the ABox via forward-chaining and must hold every derived fact in memory. Because OWL 2 RL reasoning can be mapped directly to Datalog [12], its materialization inherits Datalog's data complexity: in the worst case, the closure size scales polynomially as $O(n^v)$, where $n$ is the number of individuals and $v$ is the maximum number of distinct variables in any rule body [17]. SYNTHOLOGY, by contrast, performs *independent* backward-chaining: each sample is constructed from a single proof tree over a bounded individual pool, so the per-sample memory footprint is constant in $n$. As Table 4 shows, at small scale ($u = 3$ on OWL2Bench, roughly $10^4$ base facts) both approaches complete in comparable time. The gap becomes severe at large scale ($u = 14$, $\sim$664 K base facts): Jena's engine caches every ground goal solved during tabling and exhausts more than 100 GB of JVM heap after approximately 7.5 hours, failing to terminate under resource constraints. SYNTHOLOGY generates one million target triples over the same ontology in under 70 minutes on the same hardware, with memory usage bounded by the per-sample proof-depth cap rather than cumulative closure size.

We refer to this phenomenon as the *Reasoning Wall*: the scale at which UDM's closure-completeness goal becomes infeasible regardless of available memory. The take-

away is not that backward-chaining is universally faster than forward-chaining, but that for the practical task of *producing a fixed-size training set for a neuro-symbolic reasoner*, sampling proofs is a fundamentally better fit than materialising the full closure.

We conclude that the SYNTHOLOGY pipeline scales to a more complex ontology without modification, that the deep-hop structural advantage of targeted generation is preserved at this larger scale, and that SYNTHOLOGY avoids the *Reasoning Wall* that the UDM baseline hits at $u = 14$. As noted in Section 7, the downstream learning advantage observed on Family Tree (Experiment 2) has not been independently re-measured on OWL2Bench and is left for future work.

## 7. Threats to Validity

*Approximate Replication.* Although the SYNTHOLOGY-trained RRN consistently outperforms UDM-trained baselines, the absolute scores (PR-AUC of 60.3% in Experiment 2) indicate *approximate* rather than near-exact replication of a sound and complete symbolic reasoner. Closing this gap likely requires both more thorough RRN hyperparameter search and stronger architectures, both of which are out of scope for this paper.

*Matched-Budget Approximation.* The matched-budget protocol equalises base-fact and inferred-target counts to within $\sim 10 - 20\%$, but exact equality is unattainable because forward and backward chaining produce different closure-to-base ratios for the same input. Table 2 shows the residual mismatch is small relative to the structural gaps discussed in Section 6.

*Single Mixed Ratio.* Experiment 1 evaluates only the 50/50 mixed strategy. We do not claim the verdict "mixed is worse than pure proof-based" generalises across all blending ratios. A sweep over ratios is required to fully determine the optimal strategy, but we *did* show that proof-based negatives are valueable.

*Internal Validity.* Hyperparameters may unintentionally favour one dataset's structural distribution. The matched-budget protocol mitigates this, but does not eliminate it.

*External Validity.* SYNTHOLOGY supports a subset of OWL 2 RL; scaling to the full profile requires further empirical testing. Predictive evaluation is limited to a single neuro-symbolic architecture (the RRN) and a single ontology (Family Tree); other architectures and ontologies may respond differently to proof-based data. For OWL2Bench (Experiment 3), we report dataset-quality and scalability metrics but did not run full RRN training, so the downstream learning advantage is not yet empirically confirmed for that ontology.

*Reasoner Completeness and Hop-Depth Measurement.* The UDM baseline uses Apache Jena in a representative OWL 2 RL-aligned profile that is not fully complete; some entailed triples may therefore be absent from the materialized targets. Reported UDM hop depths are high-quality approximations.

## 8. Future Work

Future work could focus on extending the parser coverage to the remaining OWL 2 RL constructs – such as equivalence and identity axioms – or other OWL 2 profiles. Because the architecture decouples ontology parsing from proof generation, introducing new constructs requires only additional parser handlers and rule templates, while the core backward-chaining engine can remain intact.

By evaluating SYNTHOLOGY-generated data with other neuro-symbolic architectures and across a broader range of ontologies, generalizability can be tested further.

Our Experiment 1 results suggest a further direction for negative-sampling design: the naive 50/50 mixed strategy destabilises training by exposing the model to qualitatively different negatives within a single batch. A graduated curriculum – beginning with constrained negatives and transitioning to proof-based ones as training progresses – and a sweep over blending ratios are potential remedies and are left for future work.

Finally, SYNTHOLOGY could also be applied for *benchmarking* neuro-symbolic reasoners, next to its role as a training data generator.

## 9. Conclusion

We introduced SYNTHOLOGY, an ontology-agnostic synthetic data generator that replaces unguided forward-chaining materialization with targeted backward-chaining proof construction, and a proof-based negative sampling strategy that produces structurally-grounded near-miss negatives. Together, these two components address the core training-data bottleneck for neuro-symbolic reasoning: the absence of datasets that simultaneously guarantee deep multi-hop derivations and challenging negatives at scale. Our experiments demonstrate consistent gains over Unguided Deductive Materialization baselines under matched generation budgets, and feasibility on ontologies at which forward-chaining materialisation fails to terminate. We see SYNTHOLOGY as a step toward making neuro-symbolic reasoners deployable on *any* OWL 2 RL ontology, without bespoke per-ontology data engineering.

## Declaration on Generative AI

During the preparation of this manuscript, the authors used **GitHub Copilot, Google Gemini, Claude by Anthropic** in order to: **Automate cumbersome (coding) tasks, assist in their Integrated Development Environment (IDE), help with paraphrasing and rewording of the paper**.

After using these tools/services, the author reviewed and edited the content as needed and takes full responsibility for the publication's content.

## References


[1] Nenov Y, Piro R, Motik B, Horrocks I, Wu Z, Banerjee J. RDFox: A highly-scalable RDF store. In: The Semantic Web-ISWC 2015: 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part II 14. Springer; 2015. p. 3-20.

[2] Carroll JJ, Dickinson I, Dollin C, Reynolds D, Seaborne A, Wilkinson K. Jena: implementing the semantic web recommendations. In: Proceedings of the 13th international conference on World Wide Web; 2004. p. 74-83.

[3] Bonte P, Ongenae F, De Turck F. Subset reasoning for event-based systems. Ieee Access. 2019;7:107533-49.

[4] DeLong LN, Mir RF, Fleuriot JD. Neurosymbolic AI for Reasoning Over Knowledge Graphs: A Survey. IEEE Transactions on Neural Networks and Learning Systems. 2024;36(5):7822-42.

[5] Zhu X, Liu B, Zhu C, Ding Z, Yao L. Approximate Reasoning for Large-Scale ABox in OWL DL Based on Neural-Symbolic Learning. Mathematics. 2023;11(3). Available from: `https://www.mdpi.com/2227-7390/11/3/495`.

[6] Auer S, Bizer C, Kobilarov G, Lehmann J, Cyganiak R, Ives Z. DBpedia: A nucleus for a web of open data. In: The Semantic Web: 6th International Semantic Web Conference, 2nd Asian Semantic Web Conference, ISWC 2007+ ASWC 2007, Busan, Korea, November 11-15, 2007. Proceedings. Springer; 2007. p. 722-35.

[7] Bollacker K, Evans C, Paritosh P, Sturge T, Taylor J. Freebase: a collaboratively created graph database for structuring human knowledge. In: Proceedings of the 2008 ACM SIGMOD international conference on Management of data; 2008. p. 1247-50.

[8] Guo Y, Pan Z, Heflin J. LUBM: A benchmark for OWL knowledge base systems. Journal of Web Semantics. 2005;3(2-3):158-82.

[9] Ma L, Yang Y, Qiu Z, Deforeit G, Wang C, Tommila A, et al. UOBM: A user-centric ontology benchmark for OWL. In: Asian Semantic Web Conference. Springer; 2006. p. 170-84.

[10] Anke LE, Declerck T, Gromann D, Makni B, Hendler J, Gromann D, et al. Deep learning for noise-tolerant RDFS reasoning1. Semant Web. 2019 Jan;10(5):823–862. Available from: `https://doi.org/10.3233/SW-190363`.

[11] Hohenecker P, Lukasiewicz T. Ontology reasoning with deep neural networks. Journal of Artificial Intelligence Research. 2020;68:503-40.

[12] Motik B, Grau BC, Horrocks I, Wu Z, Fokoue A, Lutz C. OWL 2 Web Ontology Language Profiles. W3C Recommendation; 2009. Available from: `https://www.w3.org/TR/OWL~2-profiles/`.

[13] Madushanka T, Ichise R. Negative Sampling in Knowledge Graph Representation Learning: A Review; 2024. Available from: `https://arxiv.org/abs/2402.19195`.

[14] Bordes A, Usunier N, Garcia-Duran A, Weston J, Yakhnenko O. Translating embeddings for modeling multi-relational data. In: Advances in Neural Information Processing Systems. vol. 26; 2013. .

[15] Geirhos R, Jacobsen JH, Michaelis C, Zemel RS, Bethge M, Wichmann FA. Shortcut Learning in Deep Neural Networks. Nature Machine Intelligence. 2020;2(11):665-73.

[16] Kommineni VK, König-Ries B, Samuel S. From human experts to machines: An LLM supported approach to ontology and knowledge graph construction; 2024. Available from: `https://arxiv.org/abs/2403.08345`.

[17] Dantsin E, Eiter T, Gottlob G, Voronkov A. Complexity and expressive power of logic programming. ACM Computing Surveys (CSUR). 2001;33(3):374-425.